

For Loop (3 Questions)

1. Write a program to print the multiplication table of a given number using a for loop.

```
num = 5
for i in range(1,11):
    print(num,'*', i,"=",num*i)
```

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

2. Write a program to print all prime numbers between 1 and 100 using a for loop.

```
or num in range(2, 101):
    is_prime = True

    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
```

```
    if is_prime:
        print(num, end=" ")
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

3. Write a program to find the sum of all even numbers from 1 to n using a for loop.

```
# Input from the user
n = int(input("Enter the value of n: "))
```

```
sum_even = 0
```

```
# Loop through numbers from 1 to n
for i in range(1, n + 1):
    if i % 2 == 0:
        sum_even += i
```

```
# Display the result
print("Sum of even numbers =", sum_even)
Enter the value of n: 5
Sum of even numbers = 6
```

2.While Loop (3 Questions)

1. Write a program to reverse a given number using a while loop.

```
num = int(input("Enter a number: "))
rev = 0
```

```
while num > 0:
```

```

digit = num % 10
rev = rev * 10 + digit
num = num // 10

```

```

print("Reversed number:", rev)
Enter a number: 456
Reversed number: 654

```

2. Write a program to check whether a given number is a palindrome using a while loop.

```

num = int(input("Enter a number: "))

original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10

if original == reverse:
    print("Palindrome")
else:
    print("Not a palindrome")
Enter a number: 123
Not a palindrome

```

3. Write a program to calculate the sum of digits of a number using a while loop.

```

n = int(input("Enter a number: "))

sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i + 1

print("Sum =", sum)
Enter a number: 5
Sum = 15

```

3. Functions (5 Que)

1. Write a function to calculate the factorial of a number.

```

def factorial(n):
    fact = 1

    for i in range(1, n + 1):
        fact = fact * i

    return fact

num = int(input("Enter a number: "))
print("Factorial =", factorial(num))
Enter a number: 5
Factorial = 120

```

2. Write a function that takes a list as input and returns the largest element.

```
def largest_element(lst):
    if not lst:
        raise ValueError("The list is empty.")
    return max(lst)
```

```
numbers = [10, 45, 3, 99, 27]
print(largest_element(numbers))
99
```

3. Write a function to check whether a given string is a palindrome.

```
def is_palindrome(s):
    cleaned = ''.join(char.lower() for char in s if char.isalnum())
    return cleaned == cleaned[::-1]
```

```
text = "A man, a plan, a canal: Panama"
print(is_palindrome(text))
True
```

4. What is a function in Python? Explain its advantages.

A function is a reusable a block of code that perform a specific task

Avoid code repetition

make code readable

improve modularity

Easier Debugging

5. Differentiate between user-defined functions and built-in functions with examples.

There are 71 built-in functions in python built-in functions are predefined

User defined functions are created by the user

```
def greet():
    print("Hello python")
greet()
Hello python
```

4. OOPS (Theory – 9 Questions)

1. What is Object-Oriented Programming (OOP)?

OOP is a programming paradigm that organizes software designed around objects rather than function and logic

2. Explain the difference between Class and Object with an example.

Class is a blueprint or template using for creating objects

Object is an instance of a class

class	Object
@class method	instance method

cls parameter self keyword

3. What are the four pillars of OOP? Explain each.

OOP is a programming paradigm that organized software designed around objects rather than function and logic

Four

Inheritance

Encapsulation

Polymorphism

Data Abstraction

1. Inheritance

A. Single/Simple- child class inherit from single parent class

B. Multiple- child class inherit from more than one parent

C. Multilevel- In this inheritance happens in levels and setp by step inheritance

D. Hierarchical - Multiple child class inherit from single one parent

E. Hybrid - It is combination of inheritance Hierarchical + multiple multi-level + multiple

2. Encapsulation- Wrapping of Data

1. public x- access everywhere do not underscore prefix

2. protected _x - Define using single underscore access inside + child class

3. private __x- Define using double underscore access inside the class\

3. polymorphism

one interface many forms

1. Method overloading- same method name but different params- python does not support traditional overloading method

2. Method Overriding - Same method name same parameter

3. Using Inheritance

4. Duck Typing - It Looks like a duck quack like a duck

5. Operator Overloading- Same operator behaves different output + numbers + string concatenation + Merge List

4. Data Abstraction

Abstraction is a hiding internal implementation of details and only showing only essential features to the user

Abstract class- Abstract class must be use inherit from ABC

Abstract method- Abstract method using @abstractmethod

import abc as ABC, @abstract method

we Cannot create object in abstract class

4. What is Inheritance? Explain its types.

Inheritance used for code reusability

A. Single/Simple- child class inherit from single parent class

B. Multiple- child class inherit from more than one parent

C. Multilevel- In this inheritance happens in levels and -setp by step inheritance

D. Hierarchical - Multiple child class inherit from single one parent -one parent has common functionalities

E. Hybrid - It is combination of inheritance Hierarchical + multiple multi-level + multiple

5. What is Polymorphism? Explain Method Overriding and Method Overloading.

polymorphism means one method one function different behaviour

1. Method overloading- same method name but different parameters-

python does not support traditional overloading method

6. What is Encapsulation? What are its advantages?

Encapsulation is a wrapping of data

We Can Control for Class variables/method using access modifiers

1. Public

2. protected

3. private

Advantages

1. Security

2. Data Hiding

3. Controlled Access

7. What is Abstraction? How is it implemented in Python?

Abstraction is a hiding internal implementation of details and only showing only essential features to the user

Abstract Class

Abstract Method

Without Abstraction

1. Code Must be complex

2. Difficult to maintain

3. Higher chances of misuses

4. Unnecessary features to see the user

With Abstraction

1. Required Features to be exposed

2. Internal logic hidden

3. security

Data Abstract Class- Can Contain Abstract method(in-built), Act As a Blueprint

Abstraction Method- Has Declare Variable, No Implementation, Must be implemented in child class

we Cannot create object in abstract class

```
import abc from ABC, abstract method
```

8. What is the difference between Abstraction and Encapsulation?

Encapsulation	Abstraction
Data Hiding	Implementation Hiding

How to Data Is Protected | what functionalities are provided

Access Modifiers | Abstract Class

9. Explain the use of the super() function in Python.

Super is in-built python function use to the call method or constructor of parent base class from child class

```
Super(). init()
```

```
super(). init(arguments)
```

```
Super(). method_name()
```

1. The Call Parent Class Constructor

2. Reuse The parent Class Code

3. Avoid Rewriting Code

Super is follow mro(Method Resolution Order) Decide to What parent method or call to constructor

1. Code reusability

2. Code cleaner

3. Easier to Manage

It Is used in inheritance

It Follow MRO In Inheritance

Super(). init() initializes the parent class before child class